Simplicity SDK for Zephyr

- Getting Started
  - Overview
  - Where to Find Resources
  - Silicon Labs Development Tools
- Features
- Software and Hardware Configuration
  - Overview
  - Series 2 Clock Configuration
  - Series 2 DMA Configuration
  - Series 2 Pin Configuration
  - Series 2 Power Management
  - SiWx91x Clock Configuration
  - SiWx91x DMA Configuration
  - SiWx91x Pin Configuration
  - SiWx91x Power Management
- Bootloader
  - Overview
  - MCUboot Secure Boot for Series 2
- Release Notes
  - Overview
  - Quality Levels
- Support and Feedback

Copyright © 2025 Silicon Laboratories. All rights reserved.

Getting Started

Getting Started with Simplicity SDK for Zephyr

Zephyr OS is an open-source real-time operating system (RTOS) hosted by the Linux Foundation. It is designed for resource-constrained IoT devices and features a small footprint, modular architecture, built-in wireless protocols, security-focused design, and a hardware abstraction layer (HAL) that supports multiple CPU architectures.

Simplicity SDK for Zephyr is a customized distribution optimized for Silicon Labs Wireless Systems-on-Chips (SoCs). It builds on upstream Zephyr and includes a manifest repository with patches that enable hardware acceleration for cryptographic operations and along with additional sample applications targeting Silicon Labs devices.

Quick Start Guide

To get started with Simplicity SDK for Zephyr, follow the Zephyr Project Getting Started Guide. Instead of running `west init` to initialize a workspace based on the upstream manifest, use the following command:

```
west init -m https://github.com/SiliconLabsSoftware/zephyr-silabs
```

Alternatively, you can manually clone the repository and use `west init -l` to perform initialization from local sources.

The Zephyr Project Getting Started Guide describes how to set up the build environment, build an example, and flash it to a device.

To use Zephyr with Silicon Labs devices, you need certain pre-built libraries for radio functionality. Run the `west blobs fetch` command to download these libraries.

Resources

- Where to Find Resources
- Silicon Labs Development Tools

Copyright © 2025 Silicon Laboratories. All rights reserved.

Where to Find Resources

Where to Find Resources

Here are some helpful links to Zephyr Project and Silicon Labs repositories and documentation.

Zephyr Project Resources (upstream)

- Zephyr Project GitHub repo
- Silicon Labs board definitions
- Zephyr Project documentation

Silicon Labs Resources (downstream)

- Simplicity SDK for Zephyr GitHub

Copyright © 2025 Silicon Laboratories. All rights reserved.

Silicon Labs Development Tools

Silicon Labs Development Tools

In addition to the tools provided by the Zephyr Project, Silicon Labs offers several utilities that integrate with Zephyr and enhance development workflows.

Install Simplicity Studio 6 to access and run these tools individually.

- Simplicity Studio 6
- Simplicity Studio 6 documentation

Simplicity Device Manager

Simplicity Device Manager is a hardware management tool that centralizes the device handling in the Simplicity Studio tool suite. Use it to manage connected boards and devices.

Simplicity Commander

Simplicity Commander provides both GUI and command line access to debug features of EFM32 and SiWx91x devices. Use it to flash firmware, update kit firmware, and manage debug access.

Simplicity Commander integrates with the `west` command-line tool as a runner named `silabs_commander`. You can use it with `west flash` and other commands that flash binaries, such as the Twister test runner.

Energy Profiler

Simplicity Energy Profiler visualizes energy consumption of devices and systems. It provides real-time current consumption data correlated with the program counter, enabling advanced energy software monitoring. The tool helps developers optimize power performance in embedded systems, which is especially important for battery-operated devices. It supports analysis of individual devices, multiple devices within a single system, and networks of interacting wireless devices.

Network Analyzer

Simplicity Network Analyzer enables debugging of complex wireless systems. It captures wireless network activity for live or offline examination. More than a packet sniffer, the tool works with the data sniffer interface of Silicon Labs wireless chips to provide direct baseband radio feedback.

The tool reports detailed packet transmission and reception data without software overhead. Any PTI-enabled Silicon Labs platform can record radio activity regardless of application firmware, eliminating the need for a dedicated sniffer device.

To enable PTI output in any Zephyr application, add the `silabs-pti` snippet by appending `-S silabs-pti` to any `west build` invocation. See the silabs-pti snippet documentation for details.

```
Note:

• Simplicity Studio 6 does not support Zephyr project creation. Instead, it serves as a
  platform for accessing and launching the supported tools.
• The Network Analyzer tool is currently not supported for the SiWG917 device.
```

Copyright © 2025 Silicon Laboratories. All rights reserved.

Features

Features Supported in Simplicity SDK for Zephyr

SoCs

Support for Silicon Labs SoCs is added incrementally. See Software Quality Levels for details about support level for Series 2 and SiWx91x SoCs.

Boards

See the Silicon Labs board definitions in the Zephyr upstream documentation. To build for a specific board, pass the board name using the `-b` argument in the `west build` command.

You can use the Zephyr Supported Boards and Shields page to search for supported boards.

Wireless Protocols

On Series 2 devices, only Bluetooth Low Energy (BLE) is supported. On SiWx917, BLE and Wi-Fi are supported. Community contributions have added support for custom RAIL configurations to enable proprietary wireless protocols on Series 2.

The SiWG917 device supports both Wi-Fi and Bluetooth Low Energy (BLE) connectivity, enabling dual-protocol wireless operation. It is designed for ultra-low-power IoT applications that require seamless Wi-Fi networking alongside BLE-based control or provisioning.

See Software Quality Levels for details about the features supported for the wireless protocols.

Example Applications

Example applications are available in the Zephyr upstream repository under Zephyr upstream Samples and Demos. These provide reference implementations for supported features and hardware.

Copyright © 2025 Silicon Laboratories. All rights reserved.

Software and Hardware Configuration

Configuration for Silicon Labs Devices

Silicon Labs devices use Zephyr's standard configuration mechanisms -- Kconfig and Devicetree.

- Devicetree bindings documentation describes the contents of Devicetree nodes.
- Kconfig search provides documentation for Kconfig entries. Search terms such as `SPI.*SILABS` are useful for finding options related to specific drivers for Silicon Labs hardware.

Series 2-specific Configuration

While Devicetree and Kconfig are generic configuration interfaces, the values used may be specific to the hardware implementation:

- Series 2 Clock Configuration
- Series 2 DMA Configuration
- Series 2 Pin Configuration
- Series 2 Power Management

SiWx91x-specific Configuration

While Devicetree and Kconfig are generic configuration interfaces, the values used may be specific to the hardware implementation:

- SiWx91x Clock Configuration
- SiWx91x DMA Configuration
- SiWx91x Pin Configuration
- SiWx91x Power Management

Zephyr Kernel Thread and Scheduling Details

The Zephyr RTOS kernel has some features that differ from other RTOS kernels such as FreeRTOS or Micrium OS. Zephyr partitions the system into preemptive threads and cooperative threads.

- Cooperative threads always have scheduling priority over preemptive threads. They run until they yield or pend on a kernel object.
- Meta-IRQ threads are a special type of cooperative thread that can preempt other cooperative threads.
- The Bluetooth stack runs as several cooperative threads. On Series 2 devices, the Bluetooth Link Layer runs as a meta-IRQ thread.

Interrupt Priorities

Silicon Labs devices use the Simplicity SDK HAL, which requires that the base priority register of the ARM Cortex-M CPU is set to 3. A `BASEPRI` value of 3 means that the highest interrupt priorities (0-2) are not blocked by critical sections such as `irq_lock()` or spinlocks. See the Zephyr Project interrupt documentation for more details.

In Zephyr, the base priority is configured to be compatible with the Simplicity SDK by enabling `CONFIG_ZERO_LATENCY_IRQS` and setting `CONFIG_ZERO_LATENCY_LEVELS=2`. This is done by default in Series 2 and SiWx91x SoC Kconfig files.

Interrupt priorities are configured in Devicetree (DTS) by assigning an interrupt number and priority to the peripheral node:

```
&eusart1 {
    interrupts = <68 0>, <69 0>;
    interrupt-names = "rx", "tx";
}
```

Copyright © 2025 Silicon Laboratories. All rights reserved.

For normal interrupts, the numbers configured in Devicetree are automatically offset by the base priority offset. For example, a priority value of 0 in DTS corresponds to a priority of 3 in the ARM Nested Vectored Interrupt Controller (NVIC).

If an ISR is installed in the vector table using the `IRQ_ZERO_LATENCY` flag, the priority from DTS is used as-is. As noted above, such interrupts will not be blocked by interrupt locks.

Copyright © 2025 Silicon Laboratories. All rights reserved.

Series 2 Clock Configuration

Clock Configuration for Series 2

Zephyr uses the Clock Control driver at runtime to enable clock sources and retrieve clock frequencies. On Silicon Labs Series 2 devices, Clock Control is implemented using the Silicon Labs Clock Manager. Changing clock sources at runtime using the Clock Control API is currently not supported.

The Silicon Labs Clock Manager also handles oscillator initialization and clock tree configuration on boot. In Zephyr, the Clock Manager is configured through Devicetree.

By default, the SoC-level Devicetree configuration for the clock tree configures HFRCODPLL free-running at 19 MHz as the default high-frequency oscillator, and LFRCO at 32,768 Hz as the default low-frequency oscillator. Board-level Devicetree configuration can override this based on the availability of external oscillators.

Most Series 2 devices use a synchronous clock tree, and the radio must be clocked at the HFXO frequency. As a result, most boards enable the HFXO and configure the DPLL to lock to twice the HFXO frequency (for example, 76.8 MHz for a 38.4 MHz HFXO or 78 MHz for a 39 MHz HFXO).

If an LFXO is available on the board, it is typically enabled and selected as the low-frequency oscillator.

Configuring Clock Sources and Divisors

All nodes in the clock tree are exposed in the Devicetree `clocks` node, and their clock source and divisors can be configured using Devicetree properties:

```
/* Configure FSRCO as the clock source for IADC */
&iadcclk {
    clocks = <&fsrco>:
};
/* Configure a /8 divisor for HCLK */
&hclk {
    clock-div = <8>;
};
```

Configuring Oscillators

DPLL

The DPLL is an optional feature of the HFRCODPLL oscillator on Silicon Labs Series 2 devices. It is automatically enabled when a compatible clock source (HFXO, LFXO, or CLKIN0) is configured for the `hfrcodpll` node in Devicetree using the `clocks` property.

Configure the DPLL output frequency using the `clock-frequency` property, and set the M and N divisors using the `dpll-m` and `dpll-n` properties.

The formula for DPLL output frequency is `fout=fref*(n+1)/(m+1)`, where:

- `n` is between 300 and 4095
- `m` is between 0 and 4095

Higher values for `n` and `m` provide lower jitter but longer lock time.

Copyright © 2025 Silicon Laboratories. All rights reserved.

```
&hfrcodpll {
    clocks = <&hfxo>;
    clock-frequency = <DT_FREQ_M(78)>;
    dpll-n = <3839>;
    dpll-m = <1919>;
    dpll-edge = "fall";
    dpll-lock = "phase";
    dpll-autorecover;
};
```

See the Devicetree binding for the HFRCODPLL for documentation of all available configuration options.

LFRCO Precision Mode

Most Series 2 devices support a precision mode for the LFRCO, which periodically synchronizes the low-frequency oscillator to the HFXO. This improves timing accuracy across time and temperature.

Enable precision mode by adding the `precision-mode` property to the lfrco node in Devicetree:

```
&lfrco {
    precision-mode;
};
```

Precision mode enables certain applications that require 500 ppm precision of the low-frequency timer without the use of an external crystal oscillator, such as Bluetooth LE.

Copyright © 2025 Silicon Laboratories. All rights reserved.

Series 2 DMA Configuration

Direct Memory Access (DMA) Configuration for Series 2

Direct Memory Access (DMA) support is available in most peripheral drivers for Series 2 devices and can be enabled as needed.

The DMA API of Zephyr is documented upstream. This document describes how to configure and use DMA on Series 2 devices.

DMA Transfer Configuration

DMA configuration is performed using `struct dma_config`, which contains a linked list of `struct dma_block_config` structures that represent individual transfers.

On Series 2 devices, the DMA driver for the Linked Direct Memory Access (LDMA) peripheral creates a list of LDMA hardware descriptors (`LDMA_Descriptor_t`) from the list of `struct dma_block_config` items passed to the driver.

Each `struct dma_block_config` item requires at least one LDMA descriptor. If the block transfer count exceeds the maximum transfer size of a single descriptor (4096), multiple linked LDMA descriptors are created to represent the block.

The Kconfig option `CONFIG_DMA_MAX_DESCRIPTOR` configures the size of the memory pool used for LDMA descriptors, and is shared by all DMA users in the system.

Serial (USART)

The Serial driver for the USART peripheral, using the `silabs,usart-uart` binding, uses DMA to implement the asynchronous UART API (enabled using `CONFIG_UART_ASYNC_API`).

Prerequisites

1. Enable the LDMA Devicetree node.

```
&dma0 {
    status = "okay";
};
```

2. For UART instances that should use DMA, configure two `dmas` properties named `tx` and `rx` to connect to the appropriate LDMA peripheral request signals.

```
&usart0 {
    dmas = <&dma0 DMA_REQSEL_USART0TXBL>,
           <&dma0 DMA_REQSEL_USART0RXDATAV>;
    dma-names = "tx", "rx";
};
```

3. For UART instances that should not use DMA, do not include DMA configurations in their Devicetree node.
4. Set `CONFIG_UART_ASYNC_API` to `y`.

SPI (EUSART)

The SPI driver for the EUSART peripheral, using the `silabs,eusart-spi` binding, can optionally use DMA for both blocking and asynchronous operations.

Configuration

Copyright © 2025 Silicon Laboratories. All rights reserved.

The Kconfig option `CONFIG_SPI_SILABS_EUSART_DMA_MAX_BLOCKS` configures the maximum number of `struct dma_block_config` used to represent the transfers in a `struct spi_buf_set`.

Prerequisites

1. Enable the LDMA node in Devicetree:

```
&dma0 {
    status = "okay";
};
```

2. . For EUSART instances that should use DMA, configure two `dmas` properties named `tx` and `rx` to connect to the appropriate LDMA peripheral request signals.

```
&eusart1 {
    dmas = <&dma0 DMA_REQSEL_EUSART1TXFL>,
           <&dma0 DMA_REQSEL_EUSART1RXFL>;
    dma-names = "tx", "rx";
};
```

3. For EUSART instances that should *not* use DMA, do not include DMA configurations in their Devicetree node.
4. Set `CONFIG_SPI_SILABS_EUSART_DMA` to `y`.
   - This is selected automatically if `CONFIG_SPI_ASYNC` is `y`.
   - Set manually if using blocking operations, and DMA support is desired.

I2C

The driver for the I2C peripheral, using the `silabs,i2c` binding, can optionally use DMA for both blocking and asynchronous operations.

Prerequisites

1. Enable the LDMA Devicetree node.

```
&dma0 {
    status = "okay";
};
```

2. For I2C instances that should use DMA, configure two `dmas` properties named `tx` and `rx` to connect to the appropriate LDMA peripheral request signals.

```
&i2c0 {
    dmas = <&dma0 DMA_REQSEL_I2C0TXBL>,
           <&dma0 DMA_REQSEL_I2C0RXDATAV>;
    dma-names = "tx", "rx";
};
```

3. For I2C instances that should *not* use DMA, do not include DMA configurations in their Devicetree node.
4. Set `CONFIG_DMA` to `y`.

Flash

The on-chip flash driver, defined by the `silabs,series2-flash-controller` binding, supports optional DMA for write and read operations.

Prerequisites

1. Enable the LDMA Devicetree node.

```
&dma0 {
    status = "okay";
};
```

Copyright © 2025 Silicon Laboratories. All rights reserved.

2. The MSC must configure a `dmas` property that connects to the appropriate LDMA peripheral request signal.

```
&msc {
    dmas = <&dma0 DMA_REQSEL_MSCWDATA>;
};
```

3. Set `CONFIG_DMA` to `y`.
4. To enable DMA for write operations, set `CONFIG_SOC_FLASH_SILABS_S2_DMA_WRITE` to `y`. This option is selected automatically if the Devicetree node has a `dmas` property.
5. To use DMA for read operations, set `CONFIG_SOC_FLASH_SILABS_S2_DMA_READ` to `y`. This option must be selected manually.

Copyright © 2025 Silicon Laboratories. All rights reserved.

Series 2 Pin Configuration

Pin Configuration for Series 2

Pin Configuration in Zephyr uses the Pin Control interface. Unlike the distributed approach used in the Simplicity SDK, Zephyr centralizes pin configuration. All pinout configuration for digital signals is done in the `pinctrl` Devicetree node. Analog bus allocation is also performed through `pinctrl`.

To declare a pinout configuration for the digital bus (DBUS) signals of a peripheral, create a node in Devicetree to represent it:

```
#include <dt-bindings/pinctrl/silabs/xg24-pinctrl.h>

&pinctrl {
    eusart1_default: eusart1_default {
        group0 {
            pins = <EUSART1_TX_PC0>, <EUSART1_SCLK_PC2>, <EUSART1_CS_PC3>;
            drive-push-pull;
            output-high;
        };
        group1 {
            pins = <EUSART1_RX_PC1>;
            input-enable;
        };
    };
};
```

The names of pin configurations and groups are arbitrary; however, it is a common convention to name the pin configuration after the peripheral and its `pinctrl` state (for example, `eusart1` and `default`). Group names typically follow the format `group<n>`. Groups are useful to apply the same GPIO properties to multiple pins. The `pins` property contains a list of the pins and signals to route. Available signals for each device are available from the pinctrl `dt-bindings` header for the device and generally have the form `<peripheral>_<signal>_<pin>`.

Device drivers apply pinctrl states associated with the driver as part of initialization or power management resume/suspend:

```
&eusart1 {
    pinctrl-0 = <&eusart1_default>;
    pinctrl-names = "default";
};
```

The pinctrl driver performs the following actions:

- Configures GPIO mode corresponding to the pinctrl properties of the group.
- Assigns the port/pin to the peripheral signal in GPIO DBUS.
- Enables the DBUS route if the pin mode is anything but disabled (no pinctrl properties assigned); otherwise, disables it.

GPIO Mode ↔ Pinctrl Properties

Silicon Labs Series 2 GPIO modes map to pinctrl as follows:

Mode	Properties
DISABLED	*none or* `input-disable`
DISABLED w/pull-up	`input-disable`
	`bias-pull-up`

Copyright © 2025 Silicon Laboratories. All rights reserved.

Mode	Properties
INPUT	`input-enable`
INPUT w/filter	`input-enable`
	`silabs,input-filter`
INPUTPULL	`input-enable`
	`bias-pull-up` *or* `bias-pull-down`
INPUTPULLFILTER	`input-enable`
	`bias-pull-up` *or* `bias-pull-down`
	`silabs,input-filter`
PUSHPULL	`drive-push-pull`
	`output-high` *or* `output-low`
PUSHPULLALT	`drive-push-pull`
	`output-high` *or* `output-low`
	`silabs,alternate-port-control`
WIREDOR	`drive-open-source`
WIREDORPULLDOWN	`drive-open-source`
	`bias-pull-down`
WIREDAND	`drive-open-drain`
WIREDANDFILTER	`drive-open-drain`
	`silabs,input-filter`
WIREDANDPULLUP	`drive-open-drain`
	`bias-pull-up`
WIREDANDPULLUPFILTER	`drive-open-drain`
	`bias-pull-up`
	`silabs,input-filter`
WIREADANDALT	`drive-open-drain`
	`silabs,alternate-port-control`
WIREDANDALTFILTER	`drive-open-drain`
	`silabs,input-filter`
	`silabs,alternate-port-control`
WIREDANDALTPULLUP	`drive-open-drain`
	`bias-pull-up`
	`silabs,alternate-port-control`

Copyright © 2025 Silicon Laboratories. All rights reserved.

Mode	Properties
WIREDANDALTPULLUPFILTER	`drive-open-drain`
	`bias-pull-up`
	`silabs,input-filter`
	`silabs,alternate-port-control`

Analog Bus Allocation

In addition to digital pin routing through DBUS, the Pin Control driver handles analog bus (ABUS) allocation. There are a total of 12 analog buses - 2 for even pins and 2 for odd pins for each of the A, B and CD port groups. To allocate buses, use the `silabs,analog-bus` Devicetree property with macros from the pinctrl bindings header:

```
#include <dt-bindings/pinctrl/silabs/xg24-pinctrl.h>

&pinctrl {
    iadc0_default: iadc0_default {
        group0 {
            /* Allocate even bus 0 and odd bus 1 from GPIO port A for ADC use */
            silabs,analog-bus = <ABUS_AEVEN0_IADC0>, <ABUS_AODD1_IADC0>;
        };
    };
};

&iadc0 {
    pinctrl-0 = <&iadc0_default>;
    pinctrl-names = "default";
};
```

Copyright © 2025 Silicon Laboratories. All rights reserved.

Series 2 Power Management

Power Management for Series 2

Zephyr supports several modes of power management. See the upstream documentation for an overview of the different modes:

- `CONFIG_PM` : System Power Management - handles suspending the CPU and entering sleep states.
- `CONFIG_PM_DEVICE` : Device Power Management - handles suspending and resuming device drivers:
  - `CONFIG_PM_DEVICE_SYSTEM_MANAGED` : System-Managed Device Power Management - device drivers are suspended and resumed together with the CPU.
  - `CONFIG_PM_DEVICE_RUNTIME` : Device Runtime Power Management - device drivers have an independent lifecycle from the CPU.

On Silicon Labs devices, system-managed device power management is not considered particularly useful, because it limits the use of the rich set of autonomous peripherals capable of operating while the CPU is asleep.

System Power management

On Silicon Labs Series 2 devices, system power management is implemented using the Silicon Labs Power Manager, which acts as a backend for the Zephyr Power Management subsystem.

The Zephyr PM subsystem determines the desired sleep state based on its power management policy. This decision is passed to Silicon Labs Power Manager as an Energy Mode requirement. The Silicon Labs Power Manager performs the action of going to sleep, taking other direct requirements into account in addition to the Zephyr decision. This includes Energy Mode requirements from the radio subsystem through RAIL and any other Silicon Labs stacks and services that may be used in the future.

The Silicon Labs Power Manager may choose a shallower sleep state than Zephyr requests if another subsystem requires it.

The Silicon Labs Power Manager integrates with the Sleeptimer and HFXO Manager to optimize the sleep cycle to maximize the sleep duration while waking in time to restore required oscillators. The Sleeptimer must be used as the OS timer in Zephyr when Power Management is enabled. By default, this is the case due to the Kconfig option `CONFIG_SILABS_SLEEPTIMER_TIMER` defaulting to enabled. By default, the Sleeptimer is backed by the SYSRTC or RTCC peripheral, depending on availability.

Device Runtime Power Management

Most drivers for Silicon Labs Series 2 peripherals integrate with Device Runtime Power Management by implementing the `SUSPEND` and `RESUME` power management actions.

The `RESUME` action typically includes:

- Enabling the clock branch.
- Configuring `pinctrl` to the `default` state.
- Enabling the peripheral.

The `SUSPEND` action typically takes the opposite actions:

- Disabling the peripheral.
- Configuring `pinctrl` to the `sleep` state (if it exists).
- Disabling the clock branch.

Because Device Runtime Power Management requires correct reference counting using the `pm_device_driver_get(dev)` and `pm_device_driver_put(dev)` APIs by all subsystems using the driver, Device Runtime Power Management is disabled for Silicon Labs peripheral drivers by default, even if `CONFIG_PM_DEVICE_RUNTIME` is enabled.

Copyright © 2025 Silicon Laboratories. All rights reserved.

To opt in to Device Runtime Power Management for a specific peripheral, set the `zephyr,pm-device-runtime-auto` property on the DTS node of the peripheral. If this property is not set, the device will behave as if Device Runtime Power Management was not enabled -- the `RESUME` action will be called once at boot and the device will stay active for the lifetime of the program.

Pinctrl Sleep State

Devices that integrate with Device Runtime Power Management use the pinctrl `sleep` state if it exists in their `SUSPEND` action. This helps deallocate GPIO pins when the device is inactive -- either to save power by putting the pins in a power-saving configuration or to free pins for use by other drivers.

The `DISABLED` state is the GPIO state typically used for this purpose. It is configured by giving no additional properties in the pinctrl group other than the list of `pins` and optionally `bias-pull-up`. Configuring this state also ensures that DBUS routes are deallocated for the pin. See the Pin Configuration documentation for a full mapping between Silicon Labs GPIO modes and Zephyr pinctrl properties.

The following example demonstrates adding a pinctrl `sleep` state:

```
&pinctrl {
    eusart1_sleep: eusart1_sleep {
        group0 {
            pins = <EUSART1_TX_PC0>, <EUSART1_SCLK_PC2>, <EUSART1_CS_PC3>, <EUSART1_RX_PC1>;
            bias-pull-up;
        };
    };
};

&eusart1 {
    pinctrl-0 = <&eusart1_default>;
    pinctrl-1 = <&eusart1_sleep>;
    pinctrl-names = "default", "sleep";
};
```

Depending on the board design, the power savings of using a pinctrl sleep state may be marginal. The energy cost of executing the code that reconfigures the pinout may not be recouped by the savings during sleep. If the purpose of pinctrl sleep is energy savings, the given use case should be profiled to discover if it gives a lower total energy expenditure or not.

Copyright © 2025 Silicon Laboratories. All rights reserved.

SiWx91x Clock Configuration

Clock Configuration for SiWx91x

Zephyr uses the Clock Control driver to enable clock sources and retrieve clock frequencies at run time. On Silicon Labs SiWx91x devices, the clock control driver supports core operations such as on, off, get_rate, and set_rate.

By default, the System-on-chip (SoC)–level Devicetree configuration sets the SoC phase-locked loop (PLL) as the core clock source, fixed to 180 MHz. This value is specified in the board DTS files:

```
&cpu0 {
    clock-frequency = <180000000>;
};
```

The Interface PLL (INTF PLL) is used as the peripheral clock source. It is fixed at 160 MHz by default and cannot be overridden in the current implementation.

For additional details about the clock system, refer to the Clock Architecture section in the Reference Manual.

Devicetree Clock Binding

The clock controller is defined using the `silabs,siwx91x-clock` compatible. Peripherals reference their required clock via:

```
clocks = <&clock0 SIWX91X_CLK_UART0>;
```

Each `SIWX91X_CLK_*` value corresponds to a specific hardware clock gate controlled by the SiWx91x clock subsystem. Peripheral drivers internally manage enabling and disabling these clocks as needed.

Peripheral Clock Usage

During initialization, peripheral drivers:

1. Read the clock specifier from Devicetree.
2. Request clock activation using `clock_control_on()`.
3. The SiWx91x driver enables the corresponding hardware clock.

This ensures each peripheral receives the correct clock source during operation.

Copyright © 2025 Silicon Laboratories. All rights reserved.

SiWx91x DMA Configuration

Direct Memory Access (DMA) Configuration for SiWx91x

Direct Memory Access (DMA) support is available in Serial Peripheral Interface (SPI) and Inter-IC Sound (I2S) peripheral drivers for SiWx91x devices and can be enabled as needed. All peripheral-to-memory, memory-to-peripheral, and memory-to-memory transfer modes are supported, including scatter–gather operations.

DMA Driver Configuration

SiWx91x devices provide two DMA controllers, each with a dedicated Zephyr driver:

1. Micro Direct Memory Access (UDMA) driver
2. General-Purpose Direct Memory Access (GPDMA) driver

Driver capabilities:

1. GPDMA supports transfers up to 4095 bytes per descriptor, compared to 1024 bytes per descriptor for UDMA.
2. Multiple burst sizes are supported in GPDMA, improving throughput for high-speed peripherals.
3. Scatter–gather transfers are supported by both UDMA and GPDMA.

The Kconfig option `DMA_SILABS_SIWX91X_SG_BUFFER_COUNT` configures the maximum number of `RSI_UDMA_DESC_T` used to represent the `struct dma_block_config` in a DMA transfer.

The Kconfig option `GPDMA_SILABS_SIWX91X_DESCRIPTOR_COUNT` configures the maximum number of `RSI_GPDMA_DESC_T` used to represent the `struct dma_block_config` in a DMA transfer.

SPI (GSPI)

The SPI driver for the General Serial Peripheral Interface (GSPI) peripheral, which uses the `silabs,gspi` binding, can optionally use DMA for both blocking and asynchronous operations.

Configuration

The Kconfig option `SPI_SILABS_SIWX91X_GSPI_DMA_MAX_BLOCKS` configures the maximum number of `struct dma_block_config` used to represent the transfers in a `struct spi_buf_set`.

Prerequisites

1. Enable the GPDMA node in Devicetree:

```
&gpdma {
    status = "okay";
};
```

2. For GSPI instances that should use DMA, configure two `dmas` properties named `tx` and `rx` to connect to the appropriate GPDMA peripheral request signals.

```
&spi0 {
    dmas = <&gpdma 0 11>, <&gpdma 1 10>;
    dma-names = "tx", "rx";
};
```

Copyright © 2025 Silicon Laboratories. All rights reserved.

3. For SPI instances that should *not* use DMA, do not include DMA configurations in their Devicetree node.
4. Set `CONFIG_SPI_SILABS_SIWX91X_GSPI_DMA` to `y`.
   - This is selected automatically if `CONFIG_SPI_ASYNC` is `y`.
   - Set manually if using blocking operations, and DMA support is desired.

I2S

The driver for the I2S peripheral, using the `silabs,siwx91x-i2s` binding, uses DMA for write and read operations.

Prerequisites

1. Enable the UDMA Devicetree node.

```
&dma0 {
    status = "okay";
};
```

2. For I2S instances that should use DMA, configure two `dmas` properties named `tx` and `rx` to connect to the appropriate UDMA peripheral channels.

```
&i2s0 {
    dmas =  <&dma0 15>, <&dma0 14>;
    dma-names = "tx", "rx";
};
```

3. DMA is mandatory for I2S, and only UDMA is supported for I2S transfers.
4. Configure `I2S_SILABS_SIWX91X_DMA_MAX_BLOCKS`.

Copyright © 2025 Silicon Laboratories. All rights reserved.

SiWx91x Pin Configuration

Pin Configuration for SiWx91x

Pin configuration in Zephyr uses the Pin Control interface. Unlike the distributed approach used in the WiSeConnect SDK, Zephyr centralizes pin configuration. All pinout configuration for digital and analog signals for High-Power (HP) and Ultra-Low-Power (ULP) peripherals is done in the `pinctrl0` Devicetree node.

To configure the pinout for a peripheral, create a Devicetree node to represent it:

```
#include <zephyr/dt-bindings/pinctrl/silabs/siwx91x-pinctrl.h>

&pinctrl0 {
    ulpuart_default: ulpuart_default {
        out {
            pinmux = <ULPUART_TX_ULP11>;
        };

        in {
            pinmux = <ULPUART_RX_ULP9>;
        };
    };
};
```

The names of pin configurations and groups are arbitrary; however, it is a common convention to name the pin configuration after the peripheral and its pinctrl state (for example, `ulpuart` and `default`). Group names typically follow the format `group<n>`. Groups are useful to apply the same General-Purpose Input/Output (GPIO) properties to multiple pins. The `pins` property contains a list of the pins and signals to route. Available signals for each device are available from the pinctrl `dt-bindings` header for the device and generally have the form `<peripheral>_<signal>_<pin>`.

Device drivers apply pinctrl states associated with the driver as part of initialization or power management resume/suspend:

```
&ulpuart {
    pinctrl-0 = <&ulpuart_default>;
    pinctrl-names = "default";
};
```

The pinctrl driver performs the following actions:

- Enables the GPIO pad
- Enables the digital receiver for digital signals
- Configures the GPIO mode to route the signal to the appropriate peripheral

The Devicetree binding header `zephyr/dt-bindings/pinctrl/silabs/siwx91x-pinctrl.h` defines all valid combinations of signals and pins across HP and ULP GPIO ports.

Copyright © 2025 Silicon Laboratories. All rights reserved.

SiWx91x Power Management

Power Management for SiWx91x

Zephyr supports several modes of power management. See the upstream documentation for an overview of the different modes:

- `CONFIG_PM` : System Power Management - handles suspending the CPU and entering sleep states.
- `CONFIG_PM_DEVICE` : Device Power Management - handles suspending and resuming device drivers:
  - `CONFIG_PM_DEVICE_SYSTEM_MANAGED` : System-managed Device Power Management - device drivers are suspended and resumed together with the CPU.
  - `CONFIG_PM_DEVICE_RUNTIME` : Device Runtime Power Management - device drivers have an independent lifecycle from the CPU.

On SiWx91x devices, Device Runtime Power Management is the default when Device Power Management is selected.

System Power Management

On Silicon Labs SiWx91x devices, system power management is implemented using the WiSeConnect Power Manager, which acts as a backend for the Zephyr Power Management subsystem.

The Zephyr PM subsystem determines the desired sleep state based on its power management policy. This decision is passed to the WiSeConnect Power Manager as a power-state requirement. The WiSeConnect Power Manager performs the action of going to sleep, synchronizing the decision with the Network Processor (NWP).

Supported Power States

A subset of the power states available in SiWx91x are supported in Simplicity SDK for Zephyr.

Zephyr Power State	WiSeConnect Power State	Description
Active	PS4	CPU active. All peripherals enabled.
Runtime idle	PS4 Standby	CPU suspended. All peripherals enabled.
Suspend to idle	PS4 Sleep	CPU suspended. HP and ULP peripherals disabled.

Device Runtime Power Management

SiWx91x devices use Device Runtime Power Management with power-domain integration.

When System Power Management enters PS4 Sleep, the Power Domain driver is suspended. This invokes the `TURN_OFF` power management action on all High-Power (HP) and Ultra-Low-Power (ULP) peripheral drivers on the power domain (child devices of the power domain). Upon wakeup, the `TURN_ON` action is invoked on those devices.

The `TURN_ON` action typically includes:

- Enabling the peripheral clock
- Configuring `pinctrl` to the `default` state
- Initializing the hardware (from reset or wakeup)

The `RESUME` and `SUSPEND` actions may apply additional configuration.

Device Runtime Power Management requires correct reference counting using the `pm_device_driver_get(dev)` and `pm_device_driver_put(dev)` APIs by all subsystems that use the driver. Out of reset, devices are initialized to the `SUSPENDED` state, and a reference must be taken to wake them up. Most peripherals with transactional APIs take a reference internally as part of the transaction, ensuring that the peripheral is available for the duration of the transaction. However, for consistency, it is recommended to always take a reference to a driver before using it.

References

Copyright © 2025 Silicon Laboratories. All rights reserved.

- Upstream Power Management documentation

Copyright © 2025 Silicon Laboratories. All rights reserved.

Bootloader

Bootloader Support in Simplicity SDK for Zephyr

MCUboot

MCUboot is the default bootloader used in the Simplicity SDK for Zephyr. It provides a secure, flexible, and hardware-agnostic foundation for authenticated boot and firmware upgrade workflows on Silicon Labs Series 2 devices.

The SDK integrates MCUboot with Silicon Labs hardware features, including Secure Engine (SE) support, hardware-accelerated PSA Crypto operations, and Sysbuild-based application + bootloader builds.

Secure Boot Overview

Silicon Labs Series 2 devices support two Secure Engine variants:

- Hardware Secure Engine (HSE) — supports PSA Crypto builtin keys stored in SE OTP and hardware-accelerated ECDSA verification.
- Virtual Secure Engine (VSE) — uses an embedded public key within the MCUboot image and performs hardware-accelerated ECDSA verification using PSA Crypto transparent drivers.

Both variants enforce authenticated boot using digital signatures.

For full details, see:

- MCUboot Secure Boot for Series 2 Devices

MCUboot Features in the SDK

The Simplicity SDK for Zephyr enables:

- Secure boot with signature verification (HSE and VSE)
- Firmware image management and upgrade support
- Integration with Sysbuild for coordinated bootloader + application builds
- Hardware-accelerated signature verification on HSE and VSE devices
- Embedded-key verification on VSE-based devices

Copyright © 2025 Silicon Laboratories. All rights reserved.

MCUboot Secure Boot for Series 2

MCUboot Secure Boot for Series 2 Devices

Introduction

Overview

MCUboot is a secure bootloader for 32-bit microcontrollers that provides a common infrastructure for booting and upgrading firmware. This guide explains how to configure MCUboot with PSA Crypto builtin keys and hardware-accelerated cryptography for secure boot on Silicon Labs Series 2 devices that support the Hardware Secure Engine (HSE). It also describes how secure boot works on devices that use the Virtual Secure Engine (VSE), where MCUboot verifies images using an embedded public key and the same hardware-accelerated PSA Crypto implementation, but without PSA builtin keys.

Silicon Labs Series 2 devices support two Secure Engine (SE) variants:

Variant	PSA builtin Keys	Secure boot	Description
HSE (Hardware Secure Engine)	Yes	Yes	Full hardware key storage in SE one-time-programmable (OTP) memory
VSE (Virtual Secure Engine)	No	Yes	Software-based SE; secure boot supported but keys are embedded in firmware

Features

The following table summarizes the main behavioral differences between using PSA builtin keys on HSE devices and using an embedded key on VSE devices. Both flows provide the same high-level secure boot guarantees (signed images, integrity verification, and versioning).

Feature	HSE (Hardware Secure Engine)	VSE (Virtual Secure Engine)
Public key location	Stored securely in SE OTP memory as a PSA builtin key	Embedded in the MCUboot image (no PSA builtin key support)
Signature verification	Hardware-accelerated through PSA Crypto	Hardware-accelerated through PSA Crypto
Secure boot support	Yes (PSA Crypto builtin key flow)	Yes (embedded-key flow)

Boot Process Flow

The secure boot process is similar on both HSE and VSE devices, with key handling and PSA Crypto configuration being the main differences.

HSE-based devices (PSA builtin keys)

1. Device reset — System powers on or resets.
2. MCUboot initialization — Bootloader starts and initializes PSA Crypto with the HSE driver.
3. Key retrieval — MCUboot retrieves a PSA key handle referencing the public key stored inside the Secure Engine (for example, key ID `0x7FFF0001`).
4. Image verification — ECDSA signature verification is performed using hardware acceleration through PSA Crypto opaque drivers.
5. Boot decision:
    - Valid signature → Boot the application.
    - Invalid signature → Stay in MCUboot.

Copyright © 2025 Silicon Laboratories. All rights reserved.

VSE-only devices (embedded key)

1. Device reset — System powers on or resets.
2. MCUboot initialization — Bootloader initializes PSA Crypto with the VSE driver.
3. Key retrieval — MCUboot uses a public key compiled into the bootloader image.
4. Image verification — ECDSA signature verification is performed using hardware acceleration via PSA Crypto transparent drivers.
5. Boot decision:
    - Valid signature → Boot the application.
    - Invalid signature → Stay in MCUboot.

From the application's point of view, the behavior is identical on HSE and VSE devices: the device boots only authenticated firmware.

Prerequisites

Hardware Requirements

Secure boot with MCUboot is supported on both HSE-based and VSE-based Series 2 devices:

- Silicon Labs Series 2 device with HSE support (for example, EFR32BG29), or
- Silicon Labs Series 2 device with VSE support (for example, EFR32BG27)
- Matching development board (for example, BG29-RB4420A)

Software Requirements

- Simplicity SDK for Zephyr (Silicon Labs' Zephyr-based SDK with MCUboot integration)
- Simplicity Commander
    - Required for HSE devices (SE key provisioning and secure boot configuration)
    - Required for VSE devices to sign the MCUboot image so it is linked to the device's root of trust

---

Flash Partition Layout

MCUboot relies on a flash partition layout defined in Devicetree (DTS). The exact layout is board-specific, but Series 2 devices commonly follow a structure similar to the example below. The following layout matches the Devicetree definition for a 1 MiB flash device.

Example flash layout (1 MiB flash):

Partition	Address	Size	Purpose
`boot_partition`	0x00000000	48 KiB (0x0C000)	MCUboot bootloader
`slot0_partition`	0x0000C000	472 KiB (0x76000)	Primary application slot
`slot1_partition`	0x00082000	472 KiB (0x76000)	Secondary application slot
`storage_partition`	0x000F8000	32 KiB (0x08000)	Settings / Non-volatile Storage (NVS)

> Note: Addresses in the table are offsets from the flash base; boards commonly use a flash base of `0x08000000`.

The partition layout is configured under the `partitions` node in Devicetree. To adjust partition sizes, update the relevant partition nodes (for example, `boot_partition`, `slot0_partition`, `slot1_partition`, and `storage_partition`) in the board Devicetree or in an overlay file.

Copyright © 2025 Silicon Laboratories. All rights reserved.

Key Generation and Secure Engine Provisioning

This section covers the one-time setup steps for generating signing keys and configuring the Secure Engine for Secure boot. These steps apply to both HSE- and VSE-based Series 2 devices:

- HSE devices require provisioning of the public key into SE OTP, which is later used by MCUboot via PSA Crypto builtin keys.
- VSE devices require secure boot configuration and signing of the MCUboot image, because the VSE verifies the MCUboot image itself as part of the root of trust.
- Key generation is required in all cases because MCUboot signs and verifies application images.

Generate ECDSA Key Pair

Generate an ECDSA P-256 key pair using the MCUboot imgtool included with the Simplicity SDK for Zephyr:

```
imgtool keygen -k my-signing-key.pem -t ecdsa-p256
```

This creates a PEM file that contains both the private and public key.

> Security warning:
>
> - Development: Use test keys (for example, `test-ecckey.pem`).
> - Production: Generate unique keys and store them securely.
> - Never commit keys to version control — add `*.pem` to `.gitignore`.
> - SE OTP writes are permanent — keys cannot be changed once written on production devices.

Extract Public Key (HSE provisioning)

For HSE devices, extract the public key portion for SE provisioning:

```
imgtool getpub -k my-signing-key.pem > my-pubkey.pem
```

VSE devices do not use PSA builtin keys for MCUboot's application verification, but the MCUboot bootloader image must still be signed so that the VSE can authenticate it.

Provision Public Key to the Secure Engine (HSE only)

On HSE devices, write the public key to the Secure Engine's OTP memory so MCUboot can access it via PSA Crypto builtin key identifiers:

```
commander security writekey --sign my-pubkey.pem
```

When prompted, type: `continue`

> Warning: This operation is permanent and irreversible on production devices. Verify you're using the correct key before proceeding.

For additional background, see:

- Simplicity Commander Documentation.

Configure Secure Boot

Copyright © 2025 Silicon Laboratories. All rights reserved.

Apply security configuration to enable secure boot using Simplicity Commander:

```
commander security writeconfig --secureboot --nostore
```

When prompted, type: `continue`

Enabling secure boot configures the device's Secure Engine to verify the MCUboot bootloader (HSE and VSE). On HSE devices, this also enables MCUboot to use PSA Crypto builtin keys for application image verification.

Verify Secure Boot Status

```
commander security status
```

Expected output:

```
Secure Boot: enabled
```

Notes for VSE-based Devices

On VSE-only devices, the secure boot flow is the same from the application's point of view (signed image, version checks, verification by MCUboot), but:

- The public key used by MCUboot to verify application images is compiled into the MCUboot image instead of being stored in SE OTP.
- PSA builtin keys are not used by MCUboot, so SE key provisioning is only required for the VSE to verify the MCUboot image itself.
- The Simplicity SDK for Zephyr provides an MCUboot configuration with an embedded public key suitable for development on supported VSE boards.

For most users on VSE-based devices, it is sufficient to:

1. Use the standard Sysbuild-based flow described in Building and Flashing with Sysbuild.
2. Enable secure boot in the device's security configuration (for example, using `commander security writeconfig --secureboot`).

MCUboot verifies application images on VSE devices using hardware-accelerated PSA Crypto with the public key embedded in the bootloader image.

Building and Flashing with Sysbuild

Build MCUboot and Application

Use Sysbuild to build both MCUboot and your application together. For example, to build the blinky sample:

```
west build -b bg29_rb4420a --sysbuild samples/basic/blinky -- -DSB_CONFIG_BOOTLOADER_MCUBOOT=y
```

In this configuration:

- `--sysbuild` enables Zephyr's multi-image build system to build both MCUboot and the application.
- `SB_CONFIG_BOOTLOADER_MCUBOOT=y` enables MCUboot as the bootloader in the Sysbuild configuration.

In this flow:

- Sysbuild orchestrates the build of both images and manages the flash layout.
- MCUboot itself selects the appropriate signature verification method based on its Kconfig configuration and the device's Secure Engine capabilities (PSA builtin keys on HSE, embedded key on VSE).

Flash

After a successful build, flash the two images:

Copyright © 2025 Silicon Laboratories. All rights reserved.

```
west flash
```

This uses the board configuration and Sysbuild-generated metadata to program both MCUboot and the application to the correct flash locations.

Reset and Boot

Reset the device using either:

```
commander device reset
```

or the board's reset button/debugger. On reset, MCUboot verifies the application image and boots it if verification succeeds.

Advanced Configuration

Debug Logging

Enable MCUboot logging to observe the boot process by adding the following options to your application's `prj.conf` (or a Kconfig fragment that affects the MCUboot configuration via Sysbuild):

```
CONFIG_LOG=y
CONFIG_MCUBOOT_LOG_LEVEL_DBG=y
CONFIG_SERIAL=y
CONFIG_CONSOLE=y
CONFIG_UART_CONSOLE=y
```

These options are helpful during development and debugging.

Troubleshooting

MCUboot Fails to Verify Image

Symptoms: Device stays in MCUboot; signature verification failure in logs (if enabled).

Solutions:

1. Verify key match:

   ```
   imgtool getpub -k my-signing-key.pem
   ```

   Confirm that the public key matches the one provisioned to the Secure Engine (HSE) or embedded in MCUboot (VSE).
2. Verify Secure Engine configuration:

   ```
   commander security status
   ```

   Ensure that the signing key used by the build system is the same key whose public portion was written to SE OTP (HSE) or embedded in MCUboot (VSE).

Build Fails with PSA or Mbed TLS Errors

Symptoms: Compilation errors related to PSA Crypto or Mbed TLS.

Solutions:

1. Perform a clean build:

   ```
   west build -b bg29_rb4420a samples/basic/blinky --pristine --sysbuild -- -DSB_CONFIG_BOOTLOADER_MCUBOOT=y
   ```

Copyright © 2025 Silicon Laboratories. All rights reserved.

2. Ensure that you are using the Simplicity SDK for Zephyr.

SE Key Write Fails

Symptoms: Simplicity Commander reports an error; secure boot shows disabled.

Solutions:

1. Check device connection:

```
commander device info
```

2. Verify that the Secure Engine is not already provisioned with an incompatible key (for example, on pre-programmed devices).
3. Ensure the public key is an ECDSA P-256 key in PEM format generated by imgtool.

Wrong Key Used for Signing

Symptoms: Signature verification fails; MCUboot rejects the application.

Solutions:

1. Verify that the private key used by the build system matches the public key provisioned to the Secure Engine (HSE) or embedded in MCUboot (VSE).
2. If the wrong public key was written permanently to SE OTP on a production device, the device might not be recoverable and must be treated accordingly.

Notes for VSE-based Devices

VSE devices use an embedded public key in the MCUboot image and do not use PSA builtin keys in MCUboot. If verification fails on a VSE device:

1. Ensure the MCUboot image was rebuilt after changing signing keys.
2. Ensure the application was signed using the private key that corresponds to the embedded public key in MCUboot.
3. Confirm that secure boot is enabled in the device's security configuration by running:

```
commander security status
```

and checking that `Secure Boot: enabled` appears in the output.

Security Considerations

Key Management Best Practices

Practice	Description
Generate unique keys	Never use example or test keys in production.
Secure storage	Use hardware security modules (HSMs), key management services (KMS), or encrypted storage for private keys.
Version control	Never commit `*.pem` or `*.key` files.
Separate environments	Use different keys for development, test, and production.
Backup	Store production key backups in secure, offline locations.
Access control	Limit access to production signing keys.

References

Copyright © 2025 Silicon Laboratories. All rights reserved.

- Simplicity SDK for Zephyr Getting Started Guide
- UG162: Simplicity Commander Reference Guide
- UG489: Gecko Bootloader User's Guide

Copyright © 2025 Silicon Laboratories. All rights reserved.

Release Notes

Simplicity SDK for Zephyr Release Notes

- Simplicity SDK for Zephyr releases

Copyright © 2025 Silicon Laboratories. All rights reserved.

Quality Levels

Software Quality Levels

Since development is done in a common repository, a release of Simplicity SDK for Zephyr will include support for devices with different quality levels, as well as contain software components with different quality levels. The quality levels include

GA

	Description
Feature completeness	Complete implementation of the documented feature set.
Bug fixes	Critical bugs and security fixes will be resolved in the latest GA release as well as the development branch.
Verification	Fully verified according to test plan.
Technical support	Issues in tagged releases of Simplicity SDK for Zephyr raised by customers through Silicon Labs support channels are handled by technical support staff.

Beta

	Description
Feature completeness	Near complete implementation of the documented feature set.
Bug fixes	Bug fixes and security fixes are not guaranteed to be applied.
Verification	Incomplete verification.
Technical support	Issues in tagged releases of Simplicity SDK for Zephyr raised by customers through Silicon Labs support channels are handled by technical support staff.

Alpha

	Description
Feature completeness	Significant changes to the implementation may be made in future releases.
Bug fixes	Bug fixes and security fixes are not guaranteed to be applied.
Verification	Incomplete verification.
Technical support	Issues in tagged releases of Simplicity SDK for Zephyr raised by customers through Silicon Labs support channels are handled by technical support staff.

Not Supported

	Description
Feature completeness	An implementation may be available, but is not compatible or tested with the given device.
Bug fixes	Not available.
Verification	Not applicable.
Technical support	Not available.

Device Support

Copyright © 2025 Silicon Laboratories. All rights reserved.

GA	Beta	Alpha	Not Supported
xG22	xG21	xG23	Series 0
xG24	xG26	xG28	Series 1
xG27	SiWG917		xG25
xG29			Series 3

Feature Support

The below tables show the level of support for features in the SDK at the device series level. Even if a device series is listed below, feature support is limited by the hardware capabilities of each individual device within each series and the support level of that device as documented in the above table.

Connectivity

The following table lists the quality level for connectivity protocols that are integrated upstream and made available through Simplicity SDK for Zephyr:

Protocol	Series 2	SiWx91x
Bluetooth	GA	Beta
Bluetooth Mesh	Not supported	Not supported
Wi-Fi	-	Beta
Thread	Not supported	-

No support for additional protocols is added in Simplicity SDK for Zephyr at this time, but the RAIL library is available on Series 2 for implementation of proprietary protocols as an Alpha quality integration.

Multiprotocol and Coex

	Series 2	SiWx91x
Bluetooth + Wi-Fi	-	Alpha
Bluetooth + RAIL	Alpha	-

Bluetooth Features

	Series 2	SiWx91x
2 Mbps PHY	GA	Beta
Coded PHY	GA	Beta
Concurrent Roles	GA	Beta
Data Length Extensions	GA	Beta
Advertising Extensions	GA	Beta
L2CAP Connection Oriented Channels	GA	Beta
Legacy advertising	GA	Beta
Extended advertising	GA	Beta
Periodic advertising	GA	Beta
Passive scanning	GA	Beta
Active scanning	GA	Beta
LE privacy	GA	Beta
LE security mode 1 levels 1/2/3/4	GA	Beta

Copyright © 2025 Silicon Laboratories. All rights reserved.

	Series 2	SiWx91x
LE security mode 2	GA	Beta
LE secure connections	GA	Beta
Just works pairing	GA	Beta
Passkey entry pairing	GA	Beta
Out of Band support	GA	Beta
Enhanced ATT	GA	Beta

Wi-Fi Features

	Series 2	SiWx91x
STA Mode	-	Beta
AP Mode	-	Beta
TWT	-	Beta
Power Save	-	Beta
Socket offload	-	Alpha

Drivers

Driver	Series 2	SiWx91x
ADC	GA	Beta
Audio (PDM)	Not supported	-
Clock Control	GA	Beta
Comparator	GA	Not supported
Counter	GA	Beta
CRC	Not supported	Not supported
DAC	GA	Not supported
DMA	GA	Beta
Entropy	GA	Beta
Flash	GA	Beta
GPIO	GA	Beta
Hwinfo	GA	Not supported
I2C	GA	Beta
I2S	Not supported	Beta
MEMC	-	Beta
Opamp	-	Not supported
Pin Control	GA	Beta
Power Domain	Not supported	Beta
PWM	TIMER: GA LETIMER: GA	Beta
Retained Memory	GA	Not supported
RTC	-	Beta
Serial (UART)	EUSART: GA USART: GA	Beta

Copyright © 2025 Silicon Laboratories. All rights reserved.

Driver	Series 2	SiWx91x
SPI	EUSART: GA (controller only) USART: Alpha (controller only)	GSPI: Beta SSI: Not supported
Kernel Timer	GA	Beta
Watchdog	GA	Beta

Security

PSA Crypto

The following table lists hardware acceleration support for the PSA Crypto API, using Secure Engine on Series 2 and the NWP subsystem on SiWx91x.

	Series 2	SiWx91x
Mbed TLS 3.6	HSE: GA VSE: GA	Beta

Secure Storage

The Secure Storage subsystem implements the PSA Secure Storage APIs in the Zephyr-based application.

The following table shows the quality level of support for Secure Storage. Only GA quality implementations use a secure encryption key provider for the storage transform.

	Series 2	SiWx91x
Secure Storage	HSE: GA VSE: Alpha	Alpha

Bootloader

MCUboot

MCUboot is the default bootloader for Zephyr-based projects. It is supported on Series 2 devices with its upstream feature set plus the integration with SE builtin keys on devices with HSE.

The following table shows the quality levels of MCUboot support:

	Series 2	SiWx91x
Sysbuild integration	GA	Not supported
Secure Boot	GA	Not supported
Secure Boot with builtin key	HSE: GA VSE: Not supported	Not supported
Hardware accelerated crypto	GA	Not supported
Bootloader upgrade	Not supported	Not supported
SE upgrade	Not supported	Not supported
Application image compression	Alpha	Not supported
Application image encryption	Alpha	Not supported

Gecko Bootloader

Copyright © 2025 Silicon Laboratories. All rights reserved.

Gecko Bootloader is not part of Simplicity SDK for Zephyr, but Gecko Bootloader from Simplicity SDK may be used together with Zephyr-based applications for Series 2 devices.

The following table shows the quality levels of the integration of Gecko Bootloader from Simplicity SDK with Simplicity SDK for Zephyr.

	Series 2	SiWx91x
Sysbuild integration	Not supported	-
Application image properties	GA	-
Bootloader interface API	Not supported	-

The following table shows the quality levels of Gecko bootloader itself.

	Series 2	SiWx91x
Secure Boot with builtin key	GA	-
Hardware accelerated crypto	GA	-
Bootloader upgrade	GA	-
SE upgrade	GA	-
Upgrade image compression	GA	-
Upgrade image encryption	GA	-

SiWx91x Bootloader

SiWx91x devices have a built-in bootloader, which is compatible with Simplicity SDK for Zephyr.

The following table shows the quality levels of the integration of this bootloader:

	Series 2	SiWx91x
Build system integration	-	Beta
Bootloader interface API	-	Beta

The following table shows the quality levels of the SiWx91X Application Bootloader itself.

	Series 2	SiWx91x
Secure Boot with builtin key	-	GA
Hardware accelerated crypto	-	GA
Bootloader upgrade	-	Not supported
Application upgrade	-	GA
NWP upgrade	-	GA
Upgrade image compression	-	Not supported
Upgrade image encryption	-	GA

Copyright © 2025 Silicon Laboratories. All rights reserved.

Support and Feedback

Support and Feedback

This section explains how to get support for Zephyr-related issues, whether you're using the Simplicity SDK for Zephyr or the upstream Zephyr release. There are two ways to get help with issues and feature requests, depending on which release you are using: Silicon Labs support and upstream community support.

- When using the Silicon Labs support, you receive prompt commercial support via the Silicon Labs Community and the support case ticketing system, following standard Silicon Labs processes.
- With upstream community support, you can get assistance via GitHub issues, your own pull requests, and Discord chats. Only questions related to the upstream release of Zephyr should be asked upstream. Anything relating to Simplicity SDK for Zephyr should be directed to Silicon Labs support channels—unless the issue can also be reproduced with the vanilla upstream release.

Silicon Labs Support

The Silicon Labs support methods include the Silicon Labs Community and the support ticket system. New feature requests should be submitted through these channels to ensure management attention for prioritization.

Access at Silicon Labs Tech Support Resources.

Silicon Labs Community

The Community is open to all customers to browse technical articles and discussions. You can start your own discussion to ask about an issue. Support engineers and other customers can respond to your questions.

Access at Silicon Labs Community.

Support Tickets

Support tickets are available for major customers for personalized and private assistance, based on existing agreements.

Access at Silicon Labs Support Tickets.

Simplicity SDK for Zephyr Repository

Use this repository to:

- Create issues related to Simplicity SDK for Zephyr.

Do not use this if:

- The issue is reproducible with the upstream Zephyr release. In that case, report it upstream.

Access at Simplicity SDK for Zephyr repository.

Upstream Community Support

Simplicity SDK for Zephyr is based on an upstream release of the Zephyr OS. If an issue can be reproduced using the upstream release, it is recommended to engage with the upstream community and contribute to the open-source project.

```
Note: Zephyr is a community project, and many contributors volunteer their time. Silicon
Labs cannot guarantee the level of support provided through upstream channels.
```

Copyright © 2025 Silicon Laboratories. All rights reserved.

The Zephyr Project Repository

Use this repository to:

- Create issues and pull requests for upstream Zephyr.

Use this when:

- The issue is reproducible in a vanilla Zephyr workspace.

Who processes the issues and PRs:

- Zephyr developers around the world triage issues and pull requests. Prioritization is not guaranteed, but critical issues affecting multiple platforms may be addressed more quickly. Rare issues affecting only specific platforms or devices may take longer to resolve.

Do not use this repository when:

- You're using Simplicity SDK for Zephyr, even if you believe the issue is in the main tree forked from Zephyr. There is no guarantee the issue isn't affected by interactions between modules or patches in the fork.

Access at Zephyr Project repository.

Discord

Discord is a free communication platform that allows users to create and join communities (called Discord Servers) where they can connect with others via voice, video, and text chat. You can use it via a web browser or as an installed application.

The Zephyr Project has a Discord server: zephyrproject

There are several categories with topic-specific channels. For example, the `Vendors` category includes a `#silabs` channel with discussion threads about Silicon Labs-related Zephyr discussions: Channel: #silabs.

Other useful channels include:

- #drivers
- #training
- #samples-and-demos
- #bt
- #devicetree

As with the upstream repository, Discord discussions should be limited to topics relevant to the upstream Zephyr project. Any topics specific to Simplicity SDK for Zephyr should be taken to downstream Silicon Labs support channels.

See Beginner's Guide to Discord.

Copyright © 2025 Silicon Laboratories. All rights reserved.